

Optimisation combinatoire

2^e année du cycle supérieur — Spécialité SIQ

Problème de Bin Packing 1D

Méthodes exactes : Branch & Bound et Programmation dynamique

Réalisé par

BOUKAIOU Rayan
KHERROUBI Mohamed El Amine
ZIADI Idriss Yacine
HIMEUR Idris
DIAR Adem Abdelhafidh

1 Introduction

Le bin packing unidimensionnel consiste à ranger des objets de tailles w_i dans des bacs de capacité C en minimisant le nombre de bacs utilisés. Le problème est NP-difficile, ce qui rend une exploration exhaustive rapidement impraticable. Dans ce travail, nous proposons deux méthodes exactes : **Branch & Bound** et **programmation dynamique**. Le Branch & Bound est privilégié pour sa capacité à résoudre des instances plus grandes grâce aux bornes et à l'élagage, tandis que la programmation dynamique sert de méthode exacte adaptée aux instances de taille modérée.

2 Formulation du problème

On dispose d'objets $i \in \{1, \dots, n\}$ de tailles $w_i \in \mathbb{N}$ et de bacs identiques de capacité $C \in \mathbb{N}$. Une solution est une partition des objets en groupes (bacs) telle que, pour chaque bac, la somme des tailles des objets affectés n'excède pas C . L'objectif est de **minimiser le nombre de bacs utilisés**. Une formulation classique en programmation linéaire en nombres entiers utilise des variables $x_{ij} \in \{0, 1\}$ indiquant si l'objet i est placé dans le bac j , et $y_j \in \{0, 1\}$ indiquant si le bac j est utilisé. On obtient alors le modèle suivant :

$$\min \sum_{j=1}^n y_j$$

sous les contraintes :

$$\sum_{j=1}^n x_{ij} = 1 \quad \forall i \in \{1, \dots, n\}, \quad \sum_{i=1}^n w_i x_{ij} \leq C y_j \quad \forall j \in \{1, \dots, n\}, \quad x_{ij} \in \{0, 1\}, \quad y_j \in \{0, 1\}.$$

3 Branch & Bound

3.1 Représentation de l'arbre de recherche

La recherche construit une affectation partielle objet par objet. Un nœud représente l'état après placement des k premiers objets : on connaît le nombre de bacs déjà ouverts et leur niveau de remplissage. Le branchement au niveau $k+1$ consiste à placer l'objet courant soit dans l'un des bacs existants où il tient, soit dans un nouveau bac.

3.2 Borne supérieure : heuristique First-Fit Decreasing (FFD)

Pour initialiser la meilleure solution connue, on utilise l'heuristique **First-Fit Decreasing**. Les objets, triés au préalable par ordre décroissant de taille, sont insérés successivement dans le premier bac où ils tiennent ; si aucun bac ne convient, un nouveau bac est ouvert. Cette méthode fournit rapidement une solution réalisable, et donc une **borne supérieure** UB sur l'optimum. Au cours de la recherche, dès qu'une solution complète utilise moins de bacs que UB , on met UB à jour, ce qui renforce immédiatement l'élagage.

3.3 Bornes inférieures et arrêt précoce

Une borne inférieure simple et admissible repose sur le volume restant à placer. Si l'ensemble des objets non encore placés a une somme de tailles S_{reste} , alors au moins

$$LB_{\text{reste}} = \left\lceil \frac{S_{\text{reste}}}{C} \right\rceil$$

bacs supplémentaires seront nécessaires, même dans le meilleur des cas. Cette borne correspond à la **relaxation continue** du problème.

Avant même de démarrer la recherche, on peut calculer une borne inférieure globale

$$LB_{\text{global}} = \left\lceil \frac{\sum_{i=1}^n w_i}{C} \right\rceil.$$

Si la solution obtenue par FFD atteint LB_{global} , alors elle est optimale et la recherche peut s'arrêter immédiatement.

3.4 Évaluation du nœud et règle d'élagage

À chaque nœud, l'objectif est d'estimer de manière fiable le meilleur résultat atteignable à partir de l'état courant, afin de décider si l'exploration doit se poursuivre. On note b le nombre de bacs déjà ouverts dans l'affectation partielle, et S_{reste} la somme des tailles des objets non encore placés. On définit alors la **fonction d'évaluation** du nœud comme une borne inférieure sur le nombre total de bacs requis :

$$f(\text{nœud}) = b + LB_{\text{reste}}.$$

Cette évaluation est **admissible** : elle ne surestime jamais le nombre minimal de bacs, car elle suppose un scénario idéal pour le placement des objets restants. La règle d'élagage découle directement de cette propriété : si $f(\text{nœud}) \geq UB$, la branche est coupée, puisqu'aucun descendant de ce nœud ne pourra produire une solution **strictement meilleure** que la meilleure solution réalisable déjà connue.

3.5 Stratégie d'exploration

Nous privilégions une exploration en profondeur (**DFS**). Cette stratégie est adaptée car elle produit rapidement des solutions complètes, ce qui permet d'améliorer UB tôt dans la recherche et donc d'augmenter la capacité d'élagage. L'ordre de branchement favorise en général le remplissage des bacs existants avant l'ouverture de nouveaux bacs, afin de trouver plus vite des solutions compactes.

Une alternative naturelle serait un parcours en largeur (**BFS**), mais celui-ci nécessite de conserver simultanément un grand nombre de nœuds en mémoire, ce qui devient rapidement prohibitif. Le DFS est donc plus économe en mémoire tout en favorisant la découverte rapide de solutions complètes.

3.6 Brisure de symétrie

Les bacs étant **indistinguables**, permuter leurs indices ne change ni la faisabilité ni la valeur de l'objectif. Ainsi, si deux bacs ont la même charge (ou la même capacité restante), placer l'objet courant dans l'un ou dans l'autre produit des sous-problèmes **isomorphes** : un simple renommage des bacs conduit à la même solution. On évite cette redondance en ne testant, pour un objet donné, qu'un seul placement par valeur de charge distincte parmi les bacs candidats, ce qui réduit le facteur de branchement sans compromettre l'optimalité.

4 Programmation dynamique

(1) **Idée.** La programmation dynamique traite le bin packing comme un problème de **partition** : on cherche à décomposer l'ensemble $\{1, \dots, n\}$ des indices d'objets en sous-ensembles disjoints, chacun représentant le contenu d'un bac. Un tel sous-ensemble, noté $T \subseteq \{1, \dots, n\}$, désigne les **indices des objets placés ensemble dans un même bac** ; il est dit **faisable** si $\sum_{i \in T} w_i \leq C$, c'est-à-dire si les objets de T tiennent dans un seul bac. L'objectif est de couvrir tous les objets avec un nombre minimal de tels sous-ensembles faisables.

(2) **État.** Pour tout sous-ensemble $S \subseteq \{1, \dots, n\}$ représentant un **ensemble d'indices d'objets restant à ranger**, on définit $dp[S]$ comme le nombre minimal de bacs nécessaires pour ranger exactement les objets indexés par S , avec $dp[\emptyset] = 0$.

(3) **Transition.** Pour ranger les objets de S , on choisit un sous-ensemble faisable $T \subseteq S$ (les objets qui constitueront un bac), puis on résout récursivement le sous-problème sur $S \setminus T$ (les objets restants à placer) :

$$dp[S] = 1 + \min_{\substack{T \subseteq S \\ \sum_{i \in T} w_i \leq C}} dp[S \setminus T].$$

En pratique, on peut précalculer l'ensemble des sous-ensembles faisables

$$\mathcal{F} = \{T \subseteq \{1, \dots, n\} : \sum_{i \in T} w_i \leq C\}$$

et ne considérer dans la transition que les $T \in \mathcal{F}$ vérifiant $T \subseteq S$.

(4) **Exactitude et limites.** La méthode est exacte car elle examine toutes les décompositions possibles en bacs faisables. Elle reste toutefois limitée par son coût exponentiel (2^n états et de nombreux choix de T), ce qui la réserve aux instances de taille modérée ou à la validation de l'optimalité sur des sous-instances.

5 Conclusion

Le **Branch & Bound** constitue une méthode exacte efficace en pratique lorsqu'il combine une bonne borne supérieure initiale (FFD), une évaluation admissible au nœud (borne inférieure sur le volume restant), une exploration en profondeur et la brisure de symétrie. La **programmation dynamique** fournit également une solution exacte, particulièrement adaptée aux instances de petite à moyenne taille, mais limitée par une croissance exponentielle en mémoire et en temps. Ces deux approches garantissent l'optimalité, avec des domaines d'efficacité complémentaires.